

Computer Simulation

Lesson 6: File Operations

Takumi Miyoshi, Taku Yamazaki

Special thanks to: Dr. Pham Thi Huong

Lesson 6

File Operations

- Read / write MAT-files (load / save)
- Read / write text files / binary files
 - Opening, closing files (fopen, fclose, permissions)
 - Text files: fprintf, fgetl, fgets
 - Binary files: fread, fwrite
- Read / write Excel files (~~xlsread~~, ~~xlswrite~~)

Takumi Miyoshi / Taku Yamazaki

Lesson 6: File Operations

Introduction

- A file can be treated as an area in a permanent storage which stores information
- A file can be an input to a script or store an output for another manipulation
- Basic file types and extensions
 - Text files: .txt
 - Excel spreadsheets: .xls / .xlsx
 - MAT files: .mat
 - Binary files: .bin

Takumi Miyoshi / Taku Yamazaki

Lesson 6: File Operations

File operations

- Basic operations on files
 - Reading from files
 - Writing to files
 - Writing to a new file (from the beginning)
 - Appending (adding) to an existing file

Takumi Miyoshi / Taku Yamazaki

Lesson 6: File Operations

MAT-file (.mat file)

- MATLAB allows reading and saving variables from files
- MAT-files are files that store the names and contents of variables
- Methods for reading: **load** function
- Methods for writing: **save** function

Writing variables to a MAT File (save)

- The save command allows to
 - Write variables to a file
 - Append variables to a MAT-file
 - To save all workspace variables
save fileName or **save('fileName.mat')**
 - Save a specific variable
save fileName variableName
or **save('fileName.mat', 'variableName')**
- These commands create a new file and write to
- See the contents: **who -file filename**

Example

```
mymat = rand(3,5); %random matrix  
x = 1:5;  
y=x.^2;  
who % see all variables  
save ex1 %save all in file ex1.mat  
who -file ex1 % see all variables in file ex1.mat  
save ex2 mymat %save mymat into ex2.mat  
who -file ex2 %see the ex2.mat content
```

Appending Variables to a MAT-file

- Add variables to an existing MAT-file
save -append fileName variableName
Or **save('fileName.mat', 'variableName', '-append')**
- Without specifying a variable, all variables from the Command Window will be added to MAT-file
- If there is a variable with the same name, it is updated by the current value
- Example:
save -append ex2 x
who -file ex2

Reading from a MAT-file

- The **load** function
- To read all variables

Syntax: **load** fileName or **load**('fileName.mat')

- To read a subset of the variables

Syntax: **load** fileName variableList

Or **load**('fileName.mat','variableList')

- Example:

clear, load ex1 % all variables

clear, load ex1 x % load variable x only

Text files

- Files which contain characters encoded in a standard format such as ASCII

– Letters: A-Z, a-z

– Numbers: 0-9

– Marks: !"#\$%&'()-=^~¥|@[{]};+,:*,<.>/_

How to Work with Text Files

- Open the file
- Perform operations on file
 - Read from the file
 - Write to the file
 - Append to the file
- Close the file

Opening and Closing Files

- **fid = fopen**('fileName', 'permission string')
- **fclose(fid)** %close a specific file
- **fclose('all')** %close all open files
- Note:
 - File identifier: fid
 - fopen function returns -1 if file does not exist
 - fclose functions return 0 if the file was successfully closed

help fopen

Permission string	Brief description
rt (r)	Read (default)
wt (w)	Write as new (discard existing contents)
at (a)	Open or create a file for appending (adding) to the end of file
r+t (r+)	Read and write (do not create)
w+t (w+)	Open or create file for reading and writing (discard existing contents)
a+t (a+)	Open or create file for reading and writing or appending to the end of file

t: file is treated as a **text file (text mode)**

without t: file is treated as a **binary file (binary mode)**

Example (fopen/fclose)

```
%file_IO1.m
```

```
% Simple task to check open and close the file
```

```
% w+t: open for reading and writing, create a  
new file if the file does not exist
```

```
fid = fopen('testfile1.txt', 'w+t');
```

```
disp('Just check file')
```

```
fclose(fid);
```

Writing to Files

Writing functions	Brief description
fprintf	Write to or append to a file Write one line at a time Default: Write to the screen

fprintf

• Syntax

fprintf(fid, 'format', variables)

- If fid is not specified, the default fid = 1 will be treated
→ the output goes to the screen

Example:

```
x=89; y = 2020;
```

```
fprintf('The sum of %2d plus %2d is %2d\n', x, y, x+y)
```

```
fprintf('pi is %.4f', pi)
```

- More detail about formatting operator used for the
fprintf function → **doc fprintf**

Example (fprintf)

```
%(file_fprintf.m) open, close, and write to file
fid = fopen('testfile2.txt', 'wt');
for i = 1:5
    fprintf (fid, 'The square of %d is %d¥n', i, i.^2);
end
fclose(fid);
```

- Check the content → **type** testfile2.txt

help fprintf

Conversion character	Brief description
%d	Integer, signed base 10
%i	Integer, signed base 10
%o	Integer, unsigned base 8
%u	Integer, unsigned base 10
%x	Integer, unsigned base 16 (hexadecimal), lowercase letters a-f
%f	Floating-point number, fixed-point notation (%5.2f)
%e	Floating-point number, exponential notation (3.141593e+00)
%c	Single character
%s	String of characters

help fprintf

Escape characters	Brief description
¥b	Backspace
¥f	Form feed
¥n	New line
¥r	Carriage return
¥t	Horizontal tab
''	Single quotation mark
%%	Percent character
¥¥	Backslash
¥xN	Character to be represented by hexadecimal number N (ex. ¥x5A => 'Z')
¥N	Character to be represented by octal number N (ex. ¥132 => 'Z')

Reading from Files

Reading functions	Brief description
fscanf	Read the entire formatted data file into a matrix, converting the characters to their ASCII equivalents
textscan	Read the entire text data file and store in a cell array
fgets	Read file as a set of strings: one string per line; one line at a time Keep the newline character '¥n' if there is one at the end of the line
fgetl	Read file as a set of strings: one string per line, one line at a time Get rid of the newline character '¥n'

Example: fgetl and fgets functions

Write a simple script to read data from testfile2.txt, and display on the screen

```
fid = fopen('testfile2.txt');
while ~feof(fid)
    aline = fgetl(fid);
    fprintf('%s¥n', aline)
end
fclose(fid);
```

```
fid = fopen('testfile2.txt');
while ~feof(fid)
    aline = fgets(fid);
    fprintf('%s¥n', aline)
end
fclose(fid);
```

Takumi Miyoshi / Taku Yamazaki

Lesson 6: File Operations

Reading: fscanf

```
%create testfile5.txt
X=1:5; Y=[X; X.^2];
fid = fopen('testfile5.txt', 'wt');
fprintf(fid, '%2d %5d¥n', Y);
fclose(fid);
%reading from the file
fid = fopen('testfile5.txt', 'rt');
Z=fscanf(fid, '%d%d', [2, inf])
fclose(fid);
```

- 'inf' is used to read all file if the number of values in the file is not known
- [2,inf]: two rows, the number of columns equal to the number of lines in the file

Takumi Miyoshi / Taku Yamazaki

Lesson 6: File Operations

Binary Files

- A file is encoded using binary notation
- A binary file can be decoded directly into numbers
- Opening a binary file
- Writing / reading
- Closing

Takumi Miyoshi / Taku Yamazaki

Lesson 6: File Operations

Write to binary files

```
%file_binw
x=1:5; y =x.^2;
fid = fopen('file_bin.bin', 'w+');
fwrite(fid, [x;y], 'double');
fclose(fid);
```

- Data type: uint8 (default), double, so on

Takumi Miyoshi / Taku Yamazaki

Lesson 6: File Operations

Reading from binary files

```
%file_binr  
fid = fopen('file_bin.bin','r');  
y=fread(fid,[2,inf],'double');  
fclose(fid);  
Y
```

Maximum number of elements
to be read from the file. (inf:
maximum is infinity)

Excel Files

- A spreadsheet file has the extension .xls
- Many other programs support reading and writing Excel files
- MATLAB supports two built-in functions
 - Reading: xlsread → read from a spreadsheet file
 - Writing: xlswrite → write to a spreadsheet file
- **xlsread and xlswrite are not recommended anymore. Use readtable / readmatrix and writetable / writematrix**

Writing to/ Reading from Excel files

```
mat = randi(100,3,4)  
writematrix(mat, 'csvfile1.csv');  
writematrix(mat, 'excelfile1.xlsx', 'Sheet', 1,  
'Range', 'A1');  
read_mat1 = readmatrix('csvfile1.csv')  
read_mat2 = readmatrix('excelfile1.xlsx')
```

Excel files (2)

- readmatrix / writematrix can treat matrices so that all the elements are the same data type
- If treating mixed data, use readcell / writecell
- When treating the table, use readtable / writetable

Practice 6.1

- Write a script to check open / close file
 - Open 1 file and check whether file already exists or not
 - If the file doesn't exist, display message
 - If the file exists, display message and close the file
 - When closing the file, check whether the file closes successfully or not

Practice 6.2

- Create a file as shown in the table
- Using fscanf, fgetl to read data from the file

Character	ASCII equivalent
s	115
h	104
i	105
b	98
a	97
u	117
r	114
A	65

Thank you very much for your attendance!